

JFN — DOCUMENTO MESTRE CONSOLIDADO (spec de execução)

Para: uma IA executora (Claude Code ou equivalente) na VM Linux GCP `~/JFN`.

Propósito: documento único, autossuficiente e *mastigado* — contém visão, arquitetura, o registro de capacidades completo (embutido), e o **spec técnico passo-a-passo por onda** com arquivos, endpoints, schemas, assinaturas de função e critérios de aceite.

Outra IA deve conseguir implementar lendo só isto. **Data:** 2026-06-08. **Ambiente:**

Ubuntu 24.04, `~/JFN`, `venv ~/JFN/.venv`, sempre `PYTHONPATH=. .`

Como usar este documento: leia §0→§3 (visão/arquitetura/capabilities) uma vez; depois execute as ondas do §5 na ordem, uma branch + PR por onda, `pytest -q` antes de cada commit. Não mexer nos módulos dos sweeps SIAFE enquanto estiverem rodando (ver `HANDOFF-2026-06-07-SIAFE-SWEEP.md`).

§0. INVARIANTES (nunca violar, em qualquer onda)

1. **OB = pagamento; Empenho ≠ pagamento.** Nunca citar empenho como total pago.
2. **Honestidade de dados:** todo número marcado `REAL VS CACHE/INDISPONÍVEL + data + fonte`. Nunca fabricar.
3. **Honestidade jurídica:** indícios, nunca acusação; presunção de legitimidade dos atos. Só TCE-RJ/MP-RJ/ Judiciário afirmam ilícito. Vale também para minutas de mandato (linguagem de diligência/representação).
4. **Honestidade de mercado:** acerto direcional >80% sustentável é irreal. Reportar OOS + custos. Toda tese vira previsão registrada e é cobrada contra o realizado. Nunca prometer certeza.
5. **Estética:** nenhum entregável feio sai (padrão due diligence profissional — Kroll/Deloitte).
6. **Segurança:** credenciais só em `.env /Secrets`; `.env`, `auth.json`, sessões e índices locais (codegraph/graphify) nunca no git.
7. **SIAFE = sessão única por sistema** (lockfile); SIAFE 1 e 2 são independentes (paralelizáveis).

8. **LGPD:** dados pessoais (TSE/folha/dossiês) só para finalidade de controle público legítima.

§1. MÉRITO E VISÃO

1.1. O que o sistema é

Inteligência fiscal e anticorrupção do Estado do RJ, operada por um deputado estadual, com a cadeia completa: dado bruto → correlação OB↔SEI↔CNPJ↔sócios → detecção estatística → leitura do processo → parecer jurídico fundamentado . Em paralelo, copiloto de mercado (Massare) e jurídico (Lex). Hoje é **maduro, honesto, porém reativo e introspectivo**.

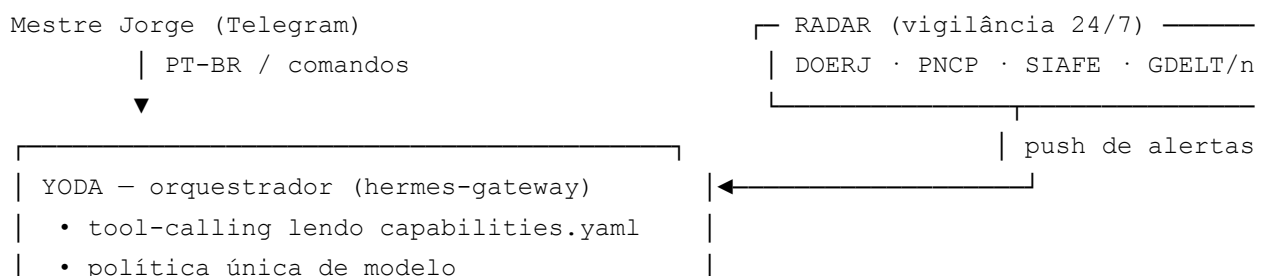
1.2. Os três saltos (a meta desta consolidação)

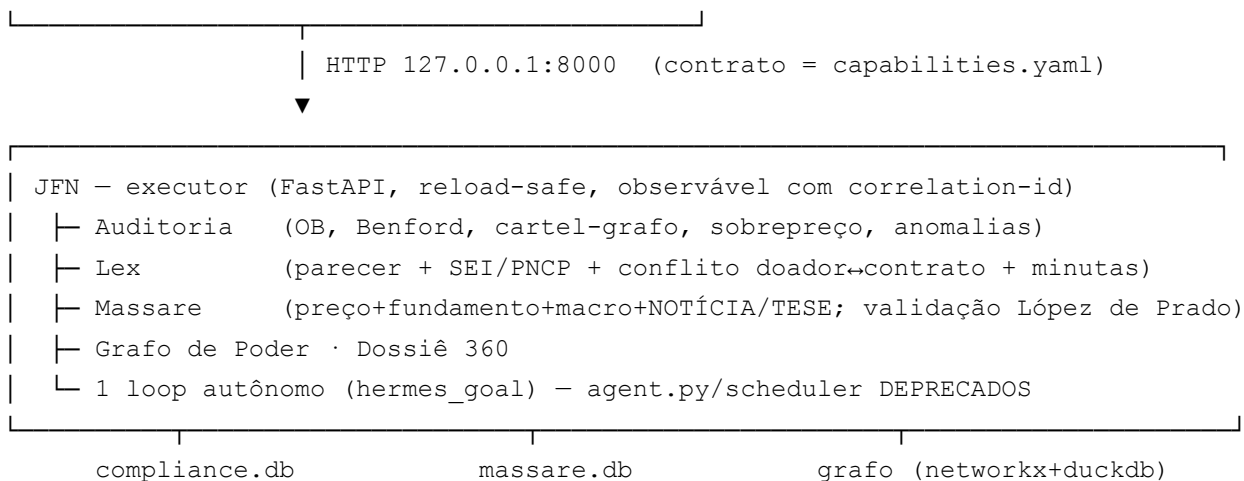
1. **Reativo → VIGILANTE:** um *Radar* observa DOERJ + PNCP + SIAFE + notícias 24/7 e dispara alertas antes de você perguntar.
2. **Fragmentado → GRAFO ÚNICO DE PODER:** sócios + servidores + doações + contratos + nomeações + parentesco num grafo persistente consultável em 1 pergunta (formaliza as investigações Amorim/Cozzolino).
3. **Diagnóstico → INSTRUMENTO DE MANDATO:** do achado à minuta (requerimento ALERJ, representação TCE-RJ, notícia de fato MP-RJ, post de transparência). É o que torna o JFN ferramenta de mandato, não só auditoria.

1.3. Correção arquitetural (base de tudo)

Não são 5 agentes. São **1 runtime (Hermes) + 1 orquestrador (Yoda) + 1 motor (JFN) com módulos de domínio (Lex, Massare, Radar, Grafo)**. Yoda orquestra · JFN executa · o resto são capacidades. Consolidar contratos > multiplicar agentes.

§2. ARQUITETURA-ALVO





2.1. Decisões transversais (Onda 0/1 — destravam o resto)

- **capabilities.yaml** = fonte única (§3). - **jfn.service reload-safe** (coletor SIAFE vira worker com lockfile próprio). - **1 loop (hermes_goal)**. - **Política única de modelo**. - **Observabilidade** (correlation-id + /api/trace/{id}). - **Hermes endurecido** (overlay idempotente pós- update).

§3. O REGISTRO DE CAPACIDADES — O QUE É E O QUE FAZER COM ELE

O que é: **capabilities.yaml** é o **contrato único** entre o orquestrador (Yoda) e o motor (JFN). Cada capacidade declara **id**, **agente**, **tipo** (**http|cli**), **rota/comando**, **descricao**, **quando_usar**, **args**, **retorno**, **status**. É a fonte de verdade que hoje está duplicada e divergente em 5 arquivos (CLAUDE.md, AMBIENTE.md, PLAYBOOK, SOUL.md, MEMORY.md).

O que fazer com ele (passo a passo):

1. **Salvar** como `~/JFN/capabilities.yaml` (raiz do repo), versionado no git.
2. **Gerar artefatos a partir dele** (nunca editar à mão os derivados):
 - `tools/gen_router_tools.py` → lê o YAML e emite as *tool-specs* (JSON schema) que o gateway do Yoda carrega para **tool-calling nativo** (cada capacidade vira uma ferramenta que o LLM pode chamar).
 - `tools/gen_capabilities_md.py` → emite a seção de capacidades do `PLAYBOOK-EXECUTOR.md` e o trecho do system prompt do Yoda. Roda no pre-commit (git hook) para nunca divergir.

3. **Validar no CI:** `tools/validate_capabilities.py` (schema + checa que toda rota `status: PRONTO` existe no `server.py`).
4. **Regra de ouro do roteador:** o Yoda **só** pode chamar `ids` presentes no registro. Ferramenta fora do registro → erro explícito ("não tenho ferramenta para isso"), **nunca invenção** (causa-raiz do bug histórico `web_search`).

O conteúdo completo do `capabilities.yaml` está embutido no §7 deste documento (copie de lá para o arquivo). Capacidades marcadas `ONDA N` são alvo das ondas correspondentes do §5.

§4. NOVIDADE DE DADOS — FONTES GRATUITAS VERIFICADAS (jun/2026)

Fonte	O que dá	Acesso	Uso no ecossistema
PNCP	Editais, contratos, atas, documentos (edital/TR)	API pública, sem login	Lex/Radar: licitação SEM depender do SEI bloqueado
Compras Dados Abertos	Preço de referência por CATMAT/CATSER	CSV/JSON + API	Lex: sobrepreço (R4). Substitui Painel de Preços (parou 04/07/2025)
TSE Dados Abertos	Doações, fornecedores, candidatos, filiação	Download aberto	Lex: conflito doador↔contrato (metodologia validada pelo TSE)
BCB/Olinda (Focus)	Expectativas Selic/IPCA/PIB/câmbio (mediana, desvio, nº)	OData, sem chave (<code>python-bcb</code>)	Massare: macro forward-looking + feature de surpresa
FRED	Macro US (CPI, PCE, NFP, Fed Funds, GDP, curva)	CSV, sem chave	Massare: calendário Fed + índice de surpresa
brapi.dev	B3 (cotação), fundamentos CVM (P/L, DY, ROE, balanços), FIIs/ETFs, PTAX, 15 macro BCB	free tier (chave grátis)	Massare: camada fundamentalista BR

Yahoo / Stooq	Cotações EOD + intraday atrasado (ações, índices, futuros contínuos , FX)	público, sem chave	Massare: motor de previsão (EOD) + boletim intraday + futuros (sinal overnight)
Websocket de exchange cripto	Cripto tempo real 24/7 (preço/trade/book)	público, sem chave/sem login	Massare: o único mercado de fato 24/7 — Binance/Coinbase/Kraken
GDELT	Notícias globais 100+ idiomas, tone , Global Knowledge Graph (temas/pessoas/locais)	DOC API + BigQuery (você tem o MCP)	Massare (China/guerra/macro diário + teses) e Radar (monitor de fornecedores/políticos)
Finnhub	Notícia de mercado, company news + sentiment, calendário econômico , earnings	free tier (chave grátis)	Massare: notícia + calendário (complementa o GDELT)

CAMADA DE DADOS 100% GRATUITA — sem IBKR, sem nada pago. Princípio: separar o plano de dados por classe de ativo e cadência, porque “24/7 real-time” só se aplica literalmente a **cripto**.

- **Cripto (24/7 real-time):** websocket público de exchange
(`wss://stream.binance.com:9443/ws` , `wss://ws-feed.exchange.coinbase.com`) — sem auth, sem custo, streaming contínuo.
- **FX (24/5):** Yahoo/Stooq (quase tempo real intraday) + **PTAX oficial** (BCB, via brapi/Olinda).
- **Ações/índices:** Yahoo/Stooq (EOD + intraday atrasado ~15 min) e **brapi.dev** (B3). Real-time tick de ações **não existe de graça em lugar nenhum** (licença de bolsa) — e o Massare não precisa: opera em horizontes de 1–63 pregões sobre fechamentos.
- **Futuros (quase 24h):** Yahoo contínuos (`ES=F` , `NQ=F` , `CL=F` , `GC=F` , `DX=F`) para sinal noturno/macro.
- **Carteira pessoal: modo manual** — holdings num arquivo/planilha (`data/carteira.json` ou Google Sheet), o sistema cruza com os dados grátis + regime/teses. Sem broker, sem credencial.

§5. SPEC TÉCNICO POR ONDA (mastigado)

Convenções: caminhos relativos a `~/JFN`. Toda rota nova entra em `server.py` (que deve estar reload-safe após Onda 0). Toda dependência nova vai em `requirements.txt` com nota de instalação CPU-only quando puxar torch. Todo coletor é **idempotente** (chave natural). Todo número marcado REAL/CACHE.

ONDA 0 — Fundação e desbloqueio

Objetivo: poder fazer deploy sem derrubar coleta + rastreabilidade + SEI acessível.

Arquivos:

- `capabilities.yaml` (raiz) + `tools/validate_capabilities.py` + hook pre-commit.
- `compliance_agent/siafe_worker.py` (novo): extrai a sessão SIAFE do processo da API para um worker independente coordenado por `data/sei_cache/siafe_lock.json`. A API só **enfileira** pedidos; o worker consome. Assim `systemctl --user restart jfn` não derruba coleta.
- `compliance_agent/obs_trace.py` (novo): middleware FastAPI que injeta `correlation_id` (uuid4) por request, loga em JSON (`logs/trace/*.jsonl`), e expõe `GET /api/trace/{id}`.
- `.env`: `SEI_PROXY_URL=` (proxy residencial BR) + manter `SEI_USER=itkava / SEI_PASS`.
Lógica do desbloqueio SEI: `collectors/sei_cdp.py` passa a usar `SEI_PROXY_URL` no contexto Playwright e prioriza o **login SIP autenticado** (sem CAPTCHA) quando o proxy estiver setado. **Aceite:** (a) `systemctl --user restart jfn` durante uma coleta não a interrompe; (b) qualquer pedido retorna header `X-Correlation-Id` e `GET /api/trace/{id}` mostra as etapas; (c) `sei_cdp` lê 1 processo via proxy sem CAPTCHA.

ONDA 1 — Orquestração

Objetivo: roteamento confiável, um cérebro só, custo previsível. **Arquivos:**

- `tools/gen_router_tools.py` (gera tool-specs do YAML) + patch no gateway do Yoda para carregá-las (substitui o roteamento por keyword). Documentar o patch em `tools/hermes_reapply_identity_patch.py` (overlay idempotente — reaplica após `hermes update`).
- `~/hermes/config.yaml`: aplicar a `politica_modelo` do §7 (fallback Gemini→Mistral, `api_max_retries:3`, `credential_pool`). Backup `.bak` antes.
- Deprecar `compliance_agent/agent.py` e loops redundantes do `scheduler.py`; **hermes_goal** é o único loop. **Aceite (4 cenários):** "relatório do fornecedor MGS" → `relatorio_inteligencia{empresa:"MGS"}`; "como está o dólar" → `massare_cenarios`; pedido ambíguo→desambigua; pedido sem capacidade→responde honestamente que

não há ferramenta. `grep -r "web_search" ~/.hermes = vazio.`

ONDA 2 — PNCP + Conflito de interesse (Lex) ★ alta prioridade

Objetivo: analisar licitação sem o SEI + a feature anticorrupção mais forte. **Arquivos:**

`compliance_agent/collectors/pncp.py` (expandir), `compliance_agent/lex_conflito.py`

(novo), rotas em `server.py`. **PNCP — endpoints (API pública, sem login):**

- **Base de consultas:** `https://pncp.gov.br/api/consulta/v1/` (e `https://pncp.gov.br/api/pncp/v1/ p/ docs`).
- **Contratações por período:** `GET /v1/contratacoes/publicacao?dataInicial=AAAAMMDD&dataFinal=AAAAMMDD&codigoModalidadeContratacao=&uf=RJ&pagina=`
- **Contratações com propostas em aberto:** `GET /v1/contratacoes/proposta?...` (insumo da fiscalização preventiva).
- **Itens/atas/contratos e documentos:** via `id` da contratação → `GET /v1/orgaos/{cnpj}/compras/{ano}/{seq}/arquivos`.
- **Implementar** `pncp.buscar_contratacoes(uf, dataIni, dataFim, modalidade=None, abertos=False) -> list[dict]` e `pncp.baixar_documentos(id_pncp) -> list[Path]` (PDF/zip → texto). Rodar os red flags R1/R7/R12 sobre o texto do edital/TR (reusar o motor do Lex). **Conflito doador↔contrato — lógica (`lex_conflito.py`):**

1. Coletar doações do TSE (`collectors/tse.py` — expandir): `prestacao_contas` por ano eleitoral → tabela `doacoes_eleitorais` (doador_cpf_cnpj, candidato, partido, valor, ano).
2. Cruzar: para cada doador-PJ ou PJ cujo **sócio (QSA)** doou, buscar contratos/OB recebidos depois.
3. Heurísticas de risco (validadas pelo TSE): empresa **constituída no ano eleitoral**; sócio **filiado** ou **parente** de candidato; contrato **acima da mediana do setor**; doador beneficiário de programa social/ renda incompatível. Cada heurística pesa; convergência = score. **Rotas:** `GET /api/pncp?uf=RJ&abertos=&cnpj=&orgao=` → `{contratacoes: [{id_pncp,objeto,valor,docs,red_flags}]}`; `GET /api/conflito?cnpj=|candidato=` → `{rede:[{doador,candidato,contrato,valor,mediana_setor,score}]}`. **Aceite:** `/api/conflito?cnpj=<um conhecido>` retorna a rede doador→contrato com score; `/api/pncp` baixa e analisa o edital de uma contratação real do RJ.

ONDA 3 — Motor de risco (fechar gap código↔documentação)

Objetivo: cada red flag R1–R12 com detector executável. **Arquivos:**

`compliance_agent/analysis/benford.py` (novo), `analysis/cartel.py` (expandir o existente), `compliance_agent/sobrepreco.py` (novo), `analysis/score_convergencia.py` (novo).

- **Benford:** `benford(valores: list[float]) -> {dist_obs, dist_esp, mad, faixa_nigrini}` (1º e 2º dígito; MAD de Nigrini com faixas conformidade). Rodar por UG e por fornecedor; alimentar score.
- **Cartel-grafo (R8):** usar o **Grafo de Poder (Onda 4)**; detectar: mesmos sócios/contador/endereço entre participantes da mesma licitação (cruzar QSA + co-endereço), rodízio de vencedores (mesma UG/objeto, vencedores alternando), propostas-cobertura (deságio do 2º < 2%). Saída em `/api/cartel`.
- **Sobrepreço (R4):** `sobrepreco(item, valor_pago) -> {mediana_ref, pct, fonte}` usando **Compras Dados Abertos** (mediana por CATMAT/CATSER) + SINAPI/EMOP/BPS por tipo de objeto.
- **Score de convergência:** soma ponderada de flags por entidade/tempo; calibrar pesos com `lex_base_empirica.py`. **Decomponível** (retornar a contribuição de cada flag). Manter `risco_achado ≠ risco_punicao`.
- **Temporal:** flag se `data_abertura_cnpj < N` meses antes da 1ª OB. **Aceite:** `/api/anomalias?ug=` inclui Benford; `/api/cartel` retorna comunidades com evidência; `/api/sobrepreco` cita o referencial e o %.

ONDA 4 — Grafo de Poder + Dossiê 360

Objetivo: o ativo investigativo central + perfil 1-comando. **Arquivos:**

`compliance_agent/grafopoder.py` (novo), `compliance_agent/dossie.py` (novo).

- **Grafo:** `construir_grafo()` une arestas de: `empresa_socios` (QSA), `registros_folha` (servidor→UG), `doacoes_eleitorais` (doador→candidato), `ordens_bancarias` /PNCP (UG→empresa), DOERJ (nomeações), co-endereço. Persistir como GraphML + espelho em DuckDB para queries rápidas. API: `grafo.vizinhanca(alvo, saltos=2, filtro="contrato") -> {nos, arestas}`; `grafo.comunidades(ug) -> [...]`.
- **Dossiê:** `dossie(alvo) -> dict` agrega cadastro (BrasilAPI), rede (grafo), contratos/OB, doações (TSE), nomeações (DOERJ), SEI/PNCP, sanções (CEIS/CNEP), notícias (GDELT) → PDF no motor da Onda 7. **Rotas:** GET `/api/grafopoder?alvo=&saltos=2&so_contrato=`; POST `/api/dossie {alvo} → {path_pdf, resumo}`. **Aceite:** "≤2 saltos do deputado X com contrato" responde; `/api/dossie` gera PDF 360°.

ONDA 5 — SEI inteligência em escala

Objetivo: “uma vez lido, buscar direcionamento”. **Arquivos:**

`compliance_agent/sei_extract.py` (extração por schema), `sei_corpus.py` (índice), `sei_direcionamento.py` (varredor). Expandir tabela `processos_sei`.

- **Extração por schema (LLM→JSON):** `extrair(texto) -> {objeto, modalidade, fundamento, valor_estimado, prazo_publicidade, exigencias_habilitacao[], n_propostas, desagios[], cnpj_participantes[], datas{}}`. Usar o modelo `raciocinio_pesado` da política, com saída JSON estrita.
- **Corpus:** SQLite FTS5 (full-text) + embeddings (modelo local grátis, ex.: `sentence-transformers CPU`) em tabela `sei_embeddings`. Função `indexar(processo)`; `buscar_semantico(consulta) -> [processos]`.
- **Detectores R1–R12 como queries** sobre os campos extraídos (ex.: R1 = marca sem “ou equivalente”; R7 = atestado/capital/localização desproporcional; R8 = `cnpj_participantes` n QSA/endereço; R12 = ETP genérico
 - `datas` comprimidas).
- **Varredor:** `varrer_direcionamento(ug=None) -> [{ref, score, red_flags, trechos}]` ranqueado. **Rota:** `GET /api/sei/direcionamento?ug=&objeto=`. **Aceite:** retorna processos suspeitos ranqueados com os red flags e os trechos-fonte.

ONDA 6 — Radar (vigilância 24/7) + Fiscalização preventiva 🌟

Objetivo: o sistema avisa antes de você perguntar. **Arquivos:** `compliance_agent/radar.py` (novo), `units systemd deploy/systemd/jfn-radar.{service,timer}`.

- **Watchlists:** tabela `radar_watch` (`alvo`, `tipo` ∈ {`cnpj`, `ug`, `nome`, `objeto`}). Rotas `POST /api/radar/vigiar`, `GET /api/radar/status`. Comandos Telegram `/vigiar`, `/radar`.
- **Ciclos (timer a cada 15–30 min):** (1) PNCP novas contratações RJ + por watchlist; pré-análise Lex; (2) **PNCP propostas em aberto** com cláusula restritiva → alerta **no prazo de impugnação** (preventivo); (3) DOERJ nomeações cruzadas com o Grafo; (4) SIAFE diário: OB de alvo monitorado / anomalia; (5) GDELT/ Finnhub: notícia sobre alvo monitorado.
- **Disparo:** alerta no Telegram (via `notifications/telegram.py`) com link/IDs e o motivo. **Aceite:** cadastrar `/vigiar <cnpj>`; ao surgir contrato/edital/menção do alvo, chega alerta no Telegram; edital aberto direcionado gera alerta dentro do prazo.

ONDA 7 — Relatório de classe mundial

Objetivo: salto estético Kroll/Deloitte. **Arquivos:**

`compliance_agent/reporting/render_html.py` (novo; Jinja2 + CSS) + conversor HTML→PDF (**WeasyPrint** ou Playwright `page.pdf()`), `reporting/charts_svg.py` (matplotlib→SVG; grafo via graphviz).

- Substituir o `fpdf2` nos relatórios de fornecedor/órgão/dossiê e no parecer Lex.
- Componentes: **rating card** de capa (score, top-3 red flags, recomendação); gráficos SVG (sparkline pagamentos/ano, heatmap P×I, barras de concentração, timeline DFD→OB, grafo societário); **proveniência por número** (REAL/CACHE+data+fonte) como nota; **score decomposto**; **hash de integridade** no rodapé. **Aceite**: relatório de fornecedor em PDF com tipografia profissional, ≥3 gráficos SVG, rating card e score explicável.

ONDA 8 — Massare: notícia, macro e fundamento 🌟 (sua prioridade)

Objetivo: fechar o buraco de “notícia de mercado relevante” + macro datada + fundamento.

Arquivos: `massare/sources.py` (expandir), `massare/news.py` (novo), `massare/calendar.py` (novo), `massare/fundamentos.py` (novo), `massare/crypto_ws.py` (novo), `massare/market_data.py` (novo — plano de dados grátis).

- **Plano de dados 100% gratuito por classe de ativo (`market_data.py` + `crypto_ws.py`):**

 - **Cripto 24/7 real-time:** `crypto_ws.py` mantém um websocket público de exchange (`wss://stream.binance.com:9443/ws/<par>@trade` e/ou Coinbase `wss://ws-feed.exchange.coinbase.com`), sem auth, gravando o último preço/trade em memória + `massare.db` . É o **único** mercado de fato 24/7.
 - **FX (24/5):** Yahoo/Stooq (intraday quase real-time) + **PTAX** oficial (brapi/Olinda).
 - **Ações/índices:** Yahoo/Stooq (EOD + atrasado ~15 min) e brapi.dev (B3). Sem real-time tick (não existe grátis; o motor não precisa — opera em fechamentos).
 - **Futuros (quase 24h):** Yahoo contínuos (`ES=F`, `NQ=F`, `CL=F`, `GC=F`, `DX=F`) para sinal noturno/macro.

- **Focus (BCB/Olinda):** `sources.focus(indicador)` via `python-bcb` (`from bcb import Expectativas`). Endpoints OData em `https://olinda.bcb.gov.br/olinda/servico/Expectativas/versao/v1/odata/` (`ExpectativasMercadoAnuais|Mensais|Trimestrais|Selic|Inflacao12Meses`). Guardar em `macro` (série `focus_*`). **Feature de surpresa** = realizado – mediana Focus.
- **Calendário macro (`calendar.py`):** **Finnhub** `/calendar/economic` (chave grátis) + FRED p/ datas de release US (CPIAUCSL, PCEPILFE, PAYEMS, FEDFUNDS, GDPC1) —

ambos grátis. Tabela `eventos_macro(data, pais, evento, impacto, previsto, anterior, atual)` . **China:** PMI/produção industrial/exportações (alto impacto p/ commodities e EWZ).

- **Notícia/sentimento (`news.py`) — o coração do que falta:**

- **GDELT** como motor primário (grátis, sem chave, 15 min, multilíngue, tone). Duas vias: **DOC 2.0 API** (<https://api.gdeltproject.org/api/v2/doc/doc?query=...&mode=ArtList&format=json>) para manchetes/tom por tema; **BigQuery** (`gdelt-bq.gdeltv2.gkg / events`) para agregação histórica e temas GKG (você tem o MCP do BigQuery). Capturar temas como produção/China, energia, conflito/guerra, política monetária.
- **Finnhub** `/news` (general/forex/crypto) + `/news-sentiment` (free tier) para complementar.
- Função `news.coletar(temas, janela) -> [{ts, titulo, fonte, tom, temas[], ativos_afetados[]}]` .

- **Fundamentos BR (`fundamentos.py`): brapi.dev** `GET /api/quote/{ticker}?modules=defaultKeyStatistics,financialData,balanceSheetHistory&fundamental=true` (P/L, DY, ROE, balanços). Tabela `fundamentos` . **Aceite:** websocket de cripto roda 24/7 e alimenta `massare.db` ; boletim diário traz **agenda macro do dia** + Focus + **3–5 manchetes com tom e ativos afetados**; `/api/massare/focus` e `/api/massare/fundamentos` respondem; feature de surpresa disponível ao modelo.

ONDA 9 — Massare: motor de TESES + validação López de Prado 🌟 (sua prioridade)

Objetivo: "conhecer, analisar e prever teses de mercado" + backtest honesto. **Arquivos:**

`massare/theses.py` (novo — o módulo-chave), `massare/validation.py` (novo), `massare/regime.py` (expandir o HMM existente p/ rotear), `massare/carteira.py` (novo).

- **Motor de teses (`theses.py`) — o que você pediu:**

1. **Ingestão→narrativa:** clusteriza as notícias (`news.py`) por tema GKG/embedding → narrativas vivas (ex.: "estímulo China", "guerra/energia", "Fed hawkish", "safra/commodities").
2. **Mapa narrativa→ativos:** tabela curada + aprendida `tese_ativo` (ex.: estímulo China → +VALE/minério/ EWZ/AUD; Fed hawkish → +DXY/–ações/–ouro).
3. **Tese testável:** `Tese(narrativa, ativos[], direcao, horizonte, confianca,`

racional) .

4. **Registro honesto:** cada tese vira `learning.record_forecast(...)` (já existe!) → cobrada contra o realizado por `grade_due()` ; `scoreboard()` mede acerto OOS por narrativa. **É assim que o sistema aprende quais teses funcionam, sem se enganar.**

5. **Saída:** `theses.atual()` -> `[{narrativa, ativos, direcao, horizonte, conf, racional, fontes[]}]` .

- **Validação (`validation.py`):** **CPCV + embargo** (além do purge atual), **Deflated Sharpe + PBO, custos/slippage SEMPRE** no retorno. Métricas: equity curve, max drawdown, turnover, Sharpe deflacionado. Backtest vetorizado com **vectorbt** (grátis).
- **Regime routing:** o HMM (`ml.py`) passa a **rotear** o ensemble (calmo-alta→momentum/trend; estresse→ mean-reversion + corte de exposição). **Meta-labeling:** secundário decide tamanho/se opera.
- **Carteira pessoal (`carteira.py`) — modo manual, sem broker:** lê posições de `data/carteira.json` (ou uma Google Sheet exportada), valoriza com os dados grátis (`market_data.py` / `crypto_ws.py`), cruza com regime/teses e alerta risco/oportunidade. Sem credencial de corretora. **Rotas:** `GET /api/massare/teses` ; `GET /api/massare/carteira` ; `massare_placar` passa a reportar DSR + Sharpe líquido. **Aceite:** `/api/massare/teses` lista teses com racional, ativos e **acerto OOS por narrativa**; backtest reporta DSR + Sharpe líquido de custos; regime roteia o ensemble.

ONDA 10 — Lex jurídico + Instrumentos de mandato

Objetivo: parecer que cita precedente + fechar o ciclo até o instrumento parlamentar.

Arquivos: `compliance_agent/lex_corpus.py` (RAG jurisprudência), `compliance_agent/mandato.py` (minutas).

- **RAG jurisprudencial:** indexar acórdãos TCU/TCE-RJ (via `lexml_fetcher.py`) com embeddings; o parecer busca e cita o **precedente mais próximo**. Dosimetria (`lex_sancoes.py`) calibrada com muitas TCE-RJ reais.
- **Minutas (`mandato.py`):** `gerar(tipo, base)` -> `docx` para `tipo` ∈ {requerimento, representacao, noticia_fato, post} , a partir de um parecer/achado. Templates em `templates/mandato/` . Cláusula de honestidade preservada (diligência/representação, nunca condenação). **Rota:** `POST /api/mandato/minuta {tipo, base}` → `{path_docx, texto}` . **Aceite:** parecer cita precedente automático; gera minuta de requerimento de informação a partir de um CNPJ.

ONDA 11 — Higiene técnica e resiliência

Objetivo: legibilidade, cobertura, robustez.

- Dividir módulos grandes (`hermes_goal 1260`, `inteligencia 1257`, `notifications/telegram 1143`, `terceirizados 1105`, `siafe_ob_orcamentaria 959`, `agent 955`, `siafe_ob 918`, `lex 903`).
- `git rm compliance_agent/collectors/sei_sei_direct.py` (0 refs — confirmar antes).
- **Fixtures de teste** do motor de risco (Benford/cartel/score) e do parser SEI (a suíte tem ~66 testes; o ponto fraco é o que produz afirmação jurídica).
- Lock SIAFE **por sistema** (`siafe_lock_{1,2}.json`); `/api/siafe/stats` com **frescor** (data da última OB por ano); **consolidação de memória** (`GET /api/memoria` lendo `Hermes + JFN MemoriaAprendizado + Massare lessons`); overlay Hermes idempotente concluído.

§6. ROADMAP RESUMIDO (ordem de execução)

0 Fundação · 1 Orquestração · **2 PNCP+Conflito** · 3 Motor de risco · 4 Grafo+Dossiê · 5 SEI · **6 Radar** · 7 Relatório · **8 Massare notícia/macro** · **9 Massare teses/validação** · 10 Lex+Mandato · 11 Higiene. (★ = maior impacto imediato. As ondas 2, 6, 8 e 9 atendem diretamente seus pedidos de Lex e Massare.)

§7. `capabilities.yaml` — CONTEÚDO COMPLETO (copiar para `~/JFN/capabilities.yaml`)

```
meta:
  versao: "2.1.0"
  atualizado: "2026-06-08"
  base_http: "http://127.0.0.1:8000"
  cli_prefixo: "cd ~/JFN && PYTHONPATH=. .venv/bin/python -m"
  invariantes:
    - "OB = pagamento; Empenho != pagamento"
    - "Honestidade: REAL vs CACHE; nunca fabricar numero"
    - "Juridico: indicios, nunca acusacao"
    - "Mercado: reportar OOS + custos; nunca prometer certeza"
    - "LGPD: dados pessoais so p/ controle publico legitimo"

capacidades:
```

---- AUDITORIA ----

- {id: relatorio_inteligencia, agente: jfn, dominio: auditoria, tipo: http, met descricao: "Due diligence de FORNECEDOR (PDF+XLSX+parecer Lex)", quando_usar args: {empresa: "string (um de empresa|cnpj)", cnpj: "string", anos: "list[int] retorno: "{cnpj,empresa,risco,score,path_pdf,path_xlsx,path_lex,grau_lex} |
- {id: relatorio_orgao, agente: jfn, dominio: auditoria, tipo: http, metodo: PO descricao: "Inteligencia de ORGAO/UG (quanto pagou e a quem)", quando_usar: args: {orgao: "string (um de orgao|ug)", ug: "string", anos: "list[int] opci
- {id: anomalias, agente: jfn, dominio: auditoria, tipo: http, metodo: GET, rot descricao: "Anomalias em OBs (+Benford na Onda 3)", quando_usar: "triagem de args: {ug: opcional, cnpj: opcional, exercicio: opcional}, retorno: "{alerta
- {id: cartel, agente: jfn, dominio: auditoria, tipo: http, metodo: GET, rota: descricao: "Cartel/bid-rigging em grafo (aprofundado Onda 3 c/ QSA)", quando args: {ug: opcional}, retorno: "{comunidades[]}"}
- {id: conflito_doador_contrato, agente: lex, dominio: auditoria, tipo: http, m descricao: "Cruza doacoes TSE com contratos/OB (doador->fornecedor)", quando args: {cnpj: "um de cnpj|candidato", candidato: "string"}, retorno: "{rede[]
- {id: sobrepreco, agente: lex, dominio: auditoria, tipo: http, metodo: GET, ro descricao: "Preco pago vs mediana (Compras Dados Abertos + SINAPI/EMOP/BPS)" args: {cnpj: opcional, item: opcional}, retorno: "{itens[{preco_pago,mediana
- {id: cruzamento, agente: jfn, dominio: auditoria, tipo: http, metodo: GET, ro descricao: "OB<->CNPJ<->SEI<->socios", quando_usar: "'cruze os dados da empr
- {id: missao_autonoma, agente: jfn, dominio: auditoria, tipo: http, metodo: PO descricao: "Auditoria autonoma aberta (loop hermes_goal CANONICO)", quando_u

---- GRAFO / DOSSIE ----

- {id: grafo_poder, agente: jfn, dominio: inteligencia, tipo: http, metodo: GET descricao: "Grafo unico: socios+servidores+doacoes+contratos+nomeacoes", qua args: {alvo: obrigatorio, saltos: "int=2", so_contrato: bool}, retorno: "{no
- {id: dossie, agente: jfn, dominio: inteligencia, tipo: http, metodo: POST, ro descricao: "Dossie 360 (cadastro+rede+contratos+doacoes+SEI/PNCP+sancoes+not

---- SEI / LICITACAO ----

- {id: ler_processo_sei, agente: lex, dominio: juridico, tipo: cli, comando: "c descricao: "Le processo SEI na integra (SIP autenticado; proxy BR)", quando_
- {id: consultar_pncp, agente: lex, dominio: juridico, tipo: http, metodo: GET, descricao: "PNCP API publica (editais/contratos/atas/DOCS) + roda R1-R12", q args: {uf: "string=RJ", orgao: opcional, cnpj: opcional, abertos: bool}, ret
- {id: buscar_direcionamento, agente: lex, dominio: juridico, tipo: http, metod descricao: "Varre corpus (SEI+PNCP) e ranqueia direcionamento (R1/R7/R8/R12)
- {id: parecer_juridico, agente: lex, dominio: juridico, tipo: cli, comando: "c descricao: "Parecer R1-R12 + base TCU/TCE-RJ (grau verde/amarelo/vermelho)",
- {id: busca_juridica, agente: lex, dominio: juridico, tipo: cli, comando: "com descricao: "Lei/acordao (TCE-RJ,TCU,LexML). RAG: Onda 10", quando_usar: "'qu
- {id: instrumento_mandato, agente: lex, dominio: juridico, tipo: http, metodo: descricao: "Minuta: requerimento(ALERJ)/representacao(TCE)/noticia de fato(M args: {tipo: "requerimento|representacao|noticia_fato|post", base: "cnpj/ug/

---- MERCADO (Massare) ----

- {id: massare_cenarios, agente: massare, dominio: mercado, tipo: http, metodo:

```

    descricao: "Snapshot multi-horizonte + regime + tom de noticias(Onda 8)", qu
- {id: massare_placar, agente: massare, dominio: mercado, tipo: http, metodo: G
    descricao: "Acuracia OOS + Sharpe liquido + DSR(Onda 9)", quando_usar: "'o M
- {id: massare_prever, agente: massare, dominio: mercado, tipo: http, metodo: P
    descricao: "Previsao direcional (walk-forward; condicionada a regime)", quan
- {id: massare_teses, agente: massare, dominio: mercado, tipo: http, metodo: GE
    descricao: "Teses de mercado: narrativa->ativos->direcao, com acerto OOS por
    args: {}, retorno: "{teses[{narrativa,ativos,direcao,horizonte,conf,racional
- {id: massare_focus, agente: massare, dominio: mercado, tipo: http, metodo: GE
    descricao: "Boletim Focus (BCB/Olinda): Selic/IPCA/PIB/cambio + variacao sem
- {id: massare_fundamentos, agente: massare, dominio: mercado, tipo: http, meto
    descricao: "Fundamentos de acao BR (P/L,DY,ROE,balancos CVM)", quando_usar:
- {id: massare_calendario, agente: massare, dominio: mercado, tipo: http, metod
    descricao: "Agenda macro (US/BR/China): CPI/NFP/FOMC/COPOM/PMI China, previs
- {id: massare_carteira, agente: massare, dominio: mercado, tipo: http, metodo:
    descricao: "Modo carteira MANUAL: holdings em data/carteira.json x macro/reg
# ---- RADAR ----
- {id: vigiar, agente: jfn, dominio: radar, tipo: http, metodo: POST, rota: /ap
    descricao: "Adiciona alvo a watchlist; alerta no Telegram em evento (PNCP/DO
- {id: radar_status, agente: jfn, dominio: radar, tipo: http, metodo: GET, rota
    descricao: "O que o Radar vigia + ultimos alertas", quando_usar: "'o que voc
# ---- COLETA / SISTEMA ----
- {id: siafe_atualizar, agente: jfn, dominio: coleta, tipo: http, metodo: POST,
    descricao: "Incremental diaria (cron 05:00); sessao unica via lockfile", qua
- {id: siafe_stats, agente: jfn, dominio: coleta, tipo: http, metodo: GET, rota
    descricao: "OBs por ano/UG (+frescor Onda 11)", quando_usar: "'quantas OBs t
- {id: siafe_coletar_ug, agente: jfn, dominio: coleta, tipo: cli, comando: "com
    descricao: "Coleta UMA UG furando teto 1000; SIAFE 1 via JFN_SIAFE_LOGIN_URL
- {id: enriquecer_socios, agente: jfn, dominio: coleta, tipo: cli, comando: "to
    descricao: "QSA (socios) via BrasilAPI; insumo do cartel-grafo e do Grafo de
- {id: status_jfn, agente: jfn, dominio: sistema, tipo: http, metodo: GET, rota
    descricao: "JFN vivo? healthcheck", quando_usar: "diagnostico", args: {}, re
- {id: trace, agente: jfn, dominio: sistema, tipo: http, metodo: GET, rota: /ap
    descricao: "Rastreia pedido ponta-a-ponta", quando_usar: "debug", args: {cor

```

politica_modelo:

```

roteamento_chat: {primario: "gemini-2.5-flash"}
raciocinio_pesado: {primario: "gemini-3-pro-preview", fallback: "gemini-2.5-pro
visao_ocr: {primario: "gemini-2.5-flash", uso: "captcha SEI, PDFs"}
fallback_chain: ["gemini-2.5-flash-lite","gemini-2.0-flash","gemini-2.0-flas
api_max_retries: 3
credential_pool: {gemini: round_robin, mistral: fill_first}

```

fontes_gratuitas:

```

pnpcp: "https://pnpcp.gov.br/api/consulta/v1/ (editais/contratos/atas/docs
compras_da: "https://www.gov.br/compras/pt-br/cidadao/compras-publicas-dados-ab
tse: "https://dadosabertos.tse.jus.br/ (doacoes/fornecedores/candidatos

```

```
focus_bcb: "https://olinda.bcb.gov.br/olinda/servico/Expectativas/versao/v1/od
fred:       "https://fred.stlouisfed.org/ (macro US; CSV sem chave)"
brapi:      "https://brapi.dev/docs (B3+fundamentos CVM+PTAX+15 macro BCB; fre
yahoo_stooq:"Yahoo Finance / Stooq (EOD + intraday atrasado: acoes/indices/fut
cripto_ws:  "wss://stream.binance.com:9443/ws + wss://ws-feed.exchange.coinba
gdelt:      "https://api.gdeltproject.org/api/v2/doc/doc + BigQuery gdelt-bq.
finnhub:    "https://finnhub.io/docs/api (news+sentiment+calendario economico;
sinapi_emop_bps: "SINAPI(obras) / EMOP-RJ(obras estaduais) / Banco de Precos em
```

§8. PENDÊNCIAS FLAGADAS

- **Credencial SIAFE 1 global:** ao chegar → `SIAFE1_USER / SIAFE1_PASS + rm data/sei_cache/ugs_siafe_1.json + rm data/sei_cache/siafe_sweep_full_1.json` (varre 2016-2023 de todas as UGs; gap atual: 2016-2018 ≠ ALERJ).
- **Stagehand (Windows):** `npm install -g @browserbasehq/stagehand → stagehand --version → concluir.`
- **Pós-sweep:** ao receber “ANÁLISE PÓS-SWEEP pronta”, ler `docs/ANALISE-POS-SWEEP-*.md` e fechar o TODO do `HANDOFF-2026-06-07-SIAFE-SWEEP.md`.
- **Chaves a obter (todas grátis):** Finnhub (token grátis), brapi.dev (token grátis para cobertura além das 4 ações de teste). Nenhuma fonte paga — cripto 24/7 via websocket público (sem chave/sem conta), Focus/FRED/GDELTYahoo/Stooq sem chave.